# OutboundAI — Complete API Integration Guide

**Version:** 1.0
**Backend:** FastAPI + LiveKit Agent + Gemini Live (Azure Container Apps)
**Purpose:** Connect any website or app to outbound AI voice calling.

---

## Table of contents


1. Overview
2. Base URL
3. Architecture
4. Integration guide
5. Authentication & limits
6. Common conventions
7. Endpoint index
8. API reference
9. Data models
10. Call outcomes
11. Supabase (database)
12. Environment variables
13. Testing with cURL
14. FAQ


---

## Overview

This Azure Application URL is the **AI calling backend**. It provides REST APIs to:

- Start outbound AI phone calls ( `POST /api/call` )
- Read call history, stats, appointments, CRM data
- Manage campaigns, agent profiles, and prompts

The same URL also serves a **legacy admin dashboard** at `/` . Integrators should **only use `/api/*` routes**.

**Do not share `.env` or Azure secrets with frontend developers.** They only need this base URL and this document.

---

## Base URL

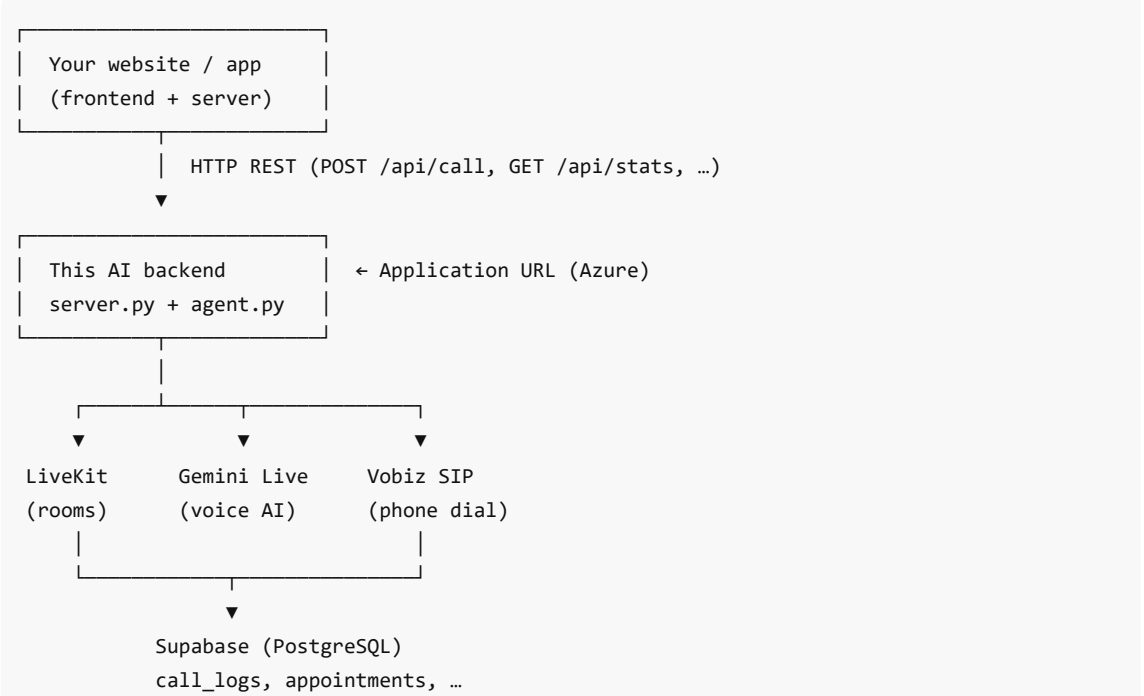
```
https://swaryx-realestate.salmonbeach-17ba963f.westus2.azurecontainerapps.io
```

| Route | Purpose |
|---|---|
| `GET /api/health` | Health check |
| `POST /api/call` | Start outbound call |
| `GET /api/calls` | Call history |
| `GET /api/stats` | Dashboard metrics |
| `GET /` | Legacy admin UI — **do not use for integration** |

Append `/api/...` to the base URL for all API calls.

---

## Architecture




**Recommended flow:** Your **server** calls this API (server-to-server). Do not call `/api/call` directly from the browser unless CORS is configured.

**What your frontend does NOT need:**

- LiveKit credentials
- Google/Gemini API keys
- Vobiz/SIP credentials
- Supabase service key

All of that runs on this backend.

---

## Integration guide

### Step 1 — Verify backend is up

```
GET /api/health
```

Expect `"status": "ok"` and `"livekit": true`.

### Step 2 — Start a test call from your server

```
const AI_API_BASE =
  "https://swaryx-realestate.salmonbeach-17ba963f.westus2.azurecontainerapps.io";
```

```
async function startCall(phone, leadName) {
  const res = await fetch(`${AI_API_BASE}/api/call`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      phone,                        // "+919876543210" E.164 required
      lead_name: leadName,
      business_name: "Swaryx",
      service_type: "property visit",
    }),
  });
  if (!res.ok) throw new Error(await res.text());
  return res.json();
}
```

### Step 3 — Poll call history / stats

```
const calls = await fetch(`${AI_API_BASE}/api/calls?page=1&limit=20`).then(r => r.json());
const stats = await fetch(`${AI_API_BASE}/api/stats`).then(r => r.json());
```

### Minimum endpoints for a new frontend

| Priority | Method | Endpoint | Purpose |
|---|---|---|---|
| 1 | `POST` | `/api/call` | Start a call |
| 2 | `GET` | `/api/calls` | Call history |
| 3 | `GET` | `/api/stats` | Dashboard metrics |
| 4 | `GET` | `/api/health` | Uptime check |

Optional: `/api/appointments`, `/api/campaigns`, `/api/agent-profiles`, `/api/crm`.

### Phone number rules

- Must be **E.164** format: starts with `+`
- India example: `+919876543210`
- Invalid format → HTTP `400`

---

## Authentication & limits

| Feature | Current status |
|---|---|
| API key / JWT | **Not implemented** — all endpoints are open |
| Rate limiting | **Not implemented** |
| Per-client quotas | **Not implemented** |
| Usage tracking | Via `call_logs` in Supabase / `GET /api/stats` |

**Production recommendation:** Add API keys and monthly call quotas before public use. Until then, restrict who knows the base URL.

**Usage limits today:** Implement on **your server** (count calls before hitting `/api/call`) or ask the AI backend team to add API key middleware.

---

## Common conventions

### Headers

```
Content-Type: application/json
```

Required for all `POST`, `PUT`, `PATCH` requests with a body.

### Success responses

- `200 OK` — read/update success
- `201` — not used explicitly; creates return `200` with `{ "status": "created", ... }`

### Error responses

FastAPI format:

```
{
  "detail": "Phone must be in E.164 format: +919876543210"
}
```

| HTTP code | Meaning |
|---|---|
| `400` | Bad request (invalid phone, missing config) |
| `403` | Forbidden (e.g. editing prompt in env-only mode) |
| `404` | Resource not found |
| `500` | Server/dispatch error |

### CORS

If your frontend runs on a **different domain** and calls the API **from the browser**, CORS must be enabled on this backend. Server-to-server calls do not need CORS.

---

## Endpoint index

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/api/health` | Health check |
| `POST` | `/api/call` | Start single outbound call |
| `GET` | `/api/calls` | List call logs (paginated) |

| | | |
|---|---|---|
| PATCH | /api/calls/{call_id}/notes | Update call notes |
| GET | /api/stats | Dashboard statistics |
| GET | /api/appointments | List appointments |
| DELETE | /api/appointments/{id} | Cancel appointment |
| GET | /api/prompt | Get system prompt |
| POST | /api/prompt | Save system prompt |
| DELETE | /api/prompt | Reset prompt to default |
| GET | /api/settings | Get config status |
| POST | /api/settings | Save settings (dev only) |
| POST | /api/setup/trunk | Create SIP trunk (admin) |
| GET | /api/logs | System logs |
| DELETE | /api/logs | Clear logs |
| GET | /api/crm | Contact list |
| GET | /api/crm/calls?phone=... | Calls for one phone |
| GET | /api/agent-profiles | List agent profiles |
| POST | /api/agent-profiles | Create agent profile |
| GET | /api/agent-profiles/{id} | Get profile |
| PUT | /api/agent-profiles/{id} | Update profile |
| DELETE | /api/agent-profiles/{id} | Delete profile |
| POST | /api/agent-profiles/{id}/set-default | Set default profile |
| POST | /api/campaigns | Create campaign |
| GET | /api/campaigns | List campaigns |
| DELETE | /api/campaigns/{id} | Delete campaign |
| POST | /api/campaigns/{id}/run | Run campaign now |
| PATCH | /api/campaigns/{id}/status | Update campaign status |
| GET | / | Legacy dashboard (ignore) |

---

## API reference

### Health

`GET /api/health`

Check backend status and core service configuration.

**Response `200` :**

```
{
  "status": "ok",
  "env_only": true,
  "supabase": true,
  "livekit": true
}
```

---

## Calls

`POST /api/call`

Start a single outbound AI voice call. **Main integration endpoint.**

**Request body:**

| Field | Type | Required | Default | Description |
|---|---|---|---|---|
| `phone` | string | **Yes** | — | E.164, e.g. `+919876543210` |
| `lead_name` | string | No | `"there"` | Lead name |
| `business_name` | string | No | `"our company"` | Company name |
| `service_type` | string | No | `"our service"` | Service type |
| `system_prompt` | string \| null | No | env/default | Override AI prompt for this call |
| `agent_profile_id` | string \| null | No | null | Agent profile UUID |

**Example request:**

```
{
  "phone": "+919876543210",
  "lead_name": "Rahul",
  "business_name": "Swaryx Real Estate",
  "service_type": "property visit",
  "system_prompt": null,
  "agent_profile_id": null
}
```

**Success `200` :**

```
{
  "status": "dispatched",
  "room": "call-919876543210-4821",
  "phone": "+919876543210"
}
```

**Errors:**

- `400` — Invalid phone or LiveKit not configured
- `500` — Dispatch failed

**Internal flow:** Creates LiveKit room → dispatches agent worker → SIP dial → Gemini Live conversation → logs result to Supabase.

---

`GET /api/calls`

Paginated call history.

**Query parameters:**

| Param | Type | Default | Description |
|---|---|---|---|
| `page` | int | `1` | Page number |
| `limit` | int | `20` | Rows per page |

**Response `200`:** Array of call log objects.

**Example:**

```
GET /api/calls?page=1&limit=20
```

---

`PATCH /api/calls/{call_id}/notes`

Update notes on a call record.

**Request body:**

```
{
  "notes": "Lead interested in 2BHK"
}
```

**Response `200`:**

```
{
  "status": "updated"
}
```

**Errors:** `404` if call not found.

---

### Stats

`GET /api/stats`

Aggregated dashboard metrics from `call_logs`.

**Response `200`:**

```
{
  "total_calls": 50,
  "booked": 12,
  "not_interested": 8,
  "avg_duration_seconds": 45.2,
  "booking_rate_percent": 24.0,
  "outcomes": {
    "booked": 12,
    "no_answer": 5,
    "not_interested": 8
  },
  "timeline": [
    { "date": "2026-05-14", "count": 2 },
    { "date": "2026-05-26", "count": 3 }
  ],
  "duration_by_outcome": {
    "booked": 120.5,
    "no_answer": 15.0
  }
}
```


---

### Appointments

`GET /api/appointments`

List booked appointments.

**Query parameters:**

| Param | Type | Description |
|---|---|---|
| `date` | string | Optional filter `YYYY-MM-DD` |

**Response `200`:** Array of appointment objects.

---

`DELETE /api/appointments/{appointment_id}`

Cancel an appointment (sets status to `cancelled`).

**Response `200`:**


```
{
  "status": "cancelled"
}
```


**Errors:** `404` if not found or already cancelled.

---

### System prompt

`GET /api/prompt`

**Response `200`:**

```
{
  "prompt": "You are Priya...",
  "is_custom": false
}
```

---

`POST /api/prompt`

Save a custom system prompt.

**Request body:**

```
{
  "prompt": "Your custom prompt text..."
}
```

**Response `200`:** `{ "status": "saved" }`

**Errors:** `403` when `CONFIG_FROM_ENV_ONLY=true` and `SYSTEM_PROMPT` is set in Azure env vars.

---

`DELETE /api/prompt`

Reset to built-in default prompt.

**Response `200`:**

```
{
  "status": "reset",
  "prompt": "..."
}
```

---

## Settings

`GET /api/settings`

Returns configuration status. Sensitive keys show `configured: true/false` without exposing values.

**Response `200` (example):**

```
{
  "env_only": true,
  "LIVEKIT_URL": { "value": "wss://...", "configured": true },
  "GEMINI_TTS_VOICE": { "value": "Leda", "configured": true },
  "GOOGLE_API_KEY": { "value": "", "configured": true }
}
```

---

`POST /api/settings`

Save settings via dashboard (local dev). Blocked for env keys in production.

**Request body:**

```
{
  "settings": {
    "ENABLED_TOOLS": "[\"book_appointment\",\"end_call\"]"
  }
}
```

---

### SIP trunk (one-time admin)

`POST /api/setup/trunk`

Creates LiveKit outbound SIP trunk using Vobiz credentials from env.

**Response `200`:**

```
{
  "status": "created",
  "trunk_id": "ST_xxxx",
  "message": "Add OUTBOUND_TRUNK_ID to your Azure environment variables and restart the
app."
}
```

---

### Logs

`GET /api/logs`

**Query parameters:**

| Param | Default | Description |
|---|---|---|
| `limit` | `200` | Max rows |
| `level` | — | `info`, `warning`, `error` |
| `source` | — | `agent`, `server` |

**Response `200`:** Array of log objects.

---

`DELETE /api/logs`

Clear all error logs.

**Response `200`:** `{ "status": "cleared" }`

---

### CRM

`GET /api/crm`

All contacts aggregated from call history.

**Response `200`:**

```
{
  "data": [
    {
      "phone_number": "+919876543210",
      "lead_name": "Rahul",
      "total_calls": 3,
      "booked": 1,
      "last_call": "2026-05-26T10:30:00",
      "last_outcome": "booked"
    }
  ]
}
```

---

`GET /api/crm/calls?phone={phone}`

Call history for one phone number.

**Query:** `phone` (required), e.g. `+919876543210`

**Response** `200` : `{ "data": [ ...call_log objects ] }`

---

### Agent profiles

Agent profiles override voice, model, prompt, and tools per call when `agent_profile_id` is passed to `/api/call` or campaigns.

`GET /api/agent-profiles`

List all profiles.

---

`POST /api/agent-profiles`

**Request body:**

| Field | Type | Default |
|---|---|---|
| `name` | string | required |
| `voice` | string | `"Leda"` |
| `model` | string | `"gemini-3.1-flash-live-preview"` |
| `system_prompt` | string \| null | null |
| `enabled_tools` | string | `"[]"` (JSON array string) |
| `is_default` | boolean | `false` |

**Example:**

```
{
  "name": "Priya - Hindi",
  "voice": "Leda",
```

```
  "model": "gemini-3.1-flash-live-preview",
  "system_prompt": null,
  "enabled_tools": "[]",
  "is_default": true
}
```

**Response `200`:**

```
{
  "status": "created",
  "id": "uuid-here"
}
```

**Available Gemini voices (examples):** `Leda`, `Kore`, `Aoede`, `Charon`, `Puck`, `Zephyr`

---

`GET /api/agent-profiles/{profile_id}`

Get one profile by ID.

---

`PUT /api/agent-profiles/{profile_id}`

Update profile (same body as create).

**Response `200`:** `{ "status": "updated" }`

---

`DELETE /api/agent-profiles/{profile_id}`

**Response `200`:** `{ "status": "deleted" }`

---

`POST /api/agent-profiles/{profile_id}/set-default`

**Response `200`:** `{ "status": "default set" }`

---

### Campaigns (bulk calling)

`POST /api/campaigns`

Create and schedule a bulk calling campaign.

**Request body:**

| Field | Type | Required | Description |
|---|---|---|---|
| `name` | string | Yes | Campaign name |
| `contacts` | array | Yes | List of contact objects |
| `schedule_type` | string | No | `"once"` \| `"daily"` \| `"weekdays"` |
| `schedule_time` | string | No | `"HH:MM"` default `"09:00"` |
| `call_delay_seconds` | int | No | Delay between calls (default `3`) |
| `system_prompt` | string \| null | No | Override prompt |

| | | | |
|---|---|---|---|
| `agent_profile_id` | string \| null | No | Profile UUID |

**Contact object:**

```
{
  "phone": "+919876543210",
  "lead_name": "Rahul",
  "business_name": "Swaryx",
  "service_type": "site visit"
}
```

**Full example:**

```
{
  "name": "Monday Leads",
  "contacts": [
    {
      "phone": "+919876543210",
      "lead_name": "Rahul",
      "business_name": "Swaryx",
      "service_type": "site visit"
    }
  ],
  "schedule_type": "once",
  "schedule_time": "09:00",
  "call_delay_seconds": 3,
  "system_prompt": null,
  "agent_profile_id": null
}
```

**Response `200` :**

```
{
  "status": "created",
  "campaign_id": "uuid",
  "campaign": { }
}
```

- `schedule_type: "once"` → runs immediately in background
- `"daily"` / `"weekdays"` → scheduled via cron

---

**`GET /api/campaigns`**

List all campaigns.

---

**`DELETE /api/campaigns/{campaign_id}`**

Delete campaign and remove scheduled job.

**Response `200` :** `{ "status": "deleted" }`

---

`POST /api/campaigns/{campaign_id}/run`

Run campaign immediately (regardless of schedule).

**Response `200`:**

```
{
  "status": "dispatching",
  "campaign_id": "uuid"
}
```

---

`PATCH /api/campaigns/{campaign_id}/status`

**Request body:**

```
{
  "status": "active"
}
```

Allowed values: `"active"` | `"paused"` | `"completed"`

---

### Legacy dashboard

`GET /`

Serves legacy OutboundAI admin UI ( `ui/index.html` ). **Do not use for integration.** Your custom frontend should only call `/api/*` .

---

## Data models

### `call_log` {#call_log}

Stored in Supabase table `call_logs` .

| Field | Type | Description |
|---|---|---|
| `id` | string (UUID) | Call record ID |
| `phone_number` | string | E.164 phone |
| `lead_name` | string \| null | Lead name |
| `outcome` | string \| null | See call outcomes |
| `reason` | string \| null | Outcome detail |
| `duration_seconds` | int \| null | Call length |
| `timestamp` | string (ISO) | When call ended |
| `recording_url` | string \| null | S3 recording URL if enabled |
| `notes` | string \| null | Manual notes |

---

### `appointment` {#appointment}

Stored in Supabase table `appointments`.

| Field | Type | Description |
|---|---|---|
| `id` | string (UUID) | Appointment ID |
| `name` | string | Lead name |
| `phone` | string | Phone number |
| `date` | string | `YYYY-MM-DD` |
| `time` | string | `HH:MM` |
| `service` | string | Service type |
| `status` | string | `booked` \| `cancelled` |
| `created_at` | string (ISO) | Created timestamp |
| `calcom_booking_uid` | string \| null | Cal.com UID if used |

---

### `campaign`

| Field | Type | Description |
|---|---|---|
| `id` | string | Campaign UUID |
| `name` | string | Name |
| `status` | string | `active` \| `paused` \| `completed` |
| `contacts_json` | string | JSON string of contacts array |
| `schedule_type` | string | `once` \| `daily` \| `weekdays` |
| `schedule_time` | string | `HH:MM` |
| `call_delay_seconds` | int | Delay between dials |
| `total_dispatched` | int | Success count |
| `total_failed` | int | Failed count |
| `agent_profile_id` | string \| null | Linked profile |

---

### `agent_profile`

| Field | Type | Description |
|---|---|---|
| `id` | string | Profile UUID |
| `name` | string | Display name |

| | | |
|---|---|---|
| `voice` | string | Gemini TTS voice |
| `model` | string | Gemini model |
| `system_prompt` | string \| null | Custom prompt |
| `enabled_tools` | string | JSON array of tool names |
| `is_default` | int | `1` = default profile |

---

## Call outcomes

Values written by the AI agent via `end_call` tool:

| Outcome | Meaning |
|---|---|
| `booked` | Appointment confirmed |
| `not_interested` | Lead declined |
| `no_answer` | No response / silence |
| `wrong_number` | Wrong person answered |
| `voicemail` | Voicemail left |
| `callback_requested` | Lead asked to call back later |

---

## Supabase (database)

This backend stores application **data** in Supabase (PostgreSQL). Credentials and AI keys stay in **Azure env vars**, not Supabase.

| Table | Purpose |
|---|---|
| `call_logs` | All call records |
| `appointments` | Bookings from AI agent |
| `contact_memory` | Lead insights across calls |
| `campaigns` | Bulk call campaigns |
| `agent_profiles` | Voice/model/prompt profiles |
| `error_logs` | System logs |
| `settings` | Local dev prompt only |

**View data:** Supabase Dashboard → **Table Editor** → select table.

**RLS:** Disabled on all tables. Backend uses service role key. Frontend should access data via these APIs, not direct Supabase (unless RLS policies are added later).

---

## Environment variables

Set on **Azure Container App** only (not in frontend code):

| Variable | Purpose |
|---|---|
| `LIVEKIT_URL`, `LIVEKIT_API_KEY`, `LIVEKIT_API_SECRET` | LiveKit connection |
| `GOOGLE_API_KEY`, `GEMINI_MODEL`, `GEMINI_TTS_VOICE` | Gemini Live AI |
| `SUPABASE_URL`, `SUPABASE_SERVICE_KEY` | Database |
| `OUTBOUND_TRUNK_ID`, `VOBIZ_*` | SIP outbound calling |
| `CONFIG_FROM_ENV_ONLY` | `true` in production |
| `SYSTEM_PROMPT` | Optional prompt override |

Optional: Twilio (SMS), S3 (recordings), Cal.com (booking sync).

---

## Testing with cURL

Replace phone with a real test number in E.164 format.

```
BASE="https://swaryx-realestate.salmonbeach-17ba963f.westus2.azurecontainerapps.io"

# Health
curl -s "$BASE/api/health" | jq

# Start call
curl -s -X POST "$BASE/api/call" \
  -H "Content-Type: application/json" \
  -d '{
    "phone": "+919876543210",
    "lead_name": "Test User",
    "business_name": "Swaryx",
    "service_type": "property visit"
  }' | jq

# Call history
curl -s "$BASE/api/calls?page=1&limit=10" | jq

# Stats
curl -s "$BASE/api/stats" | jq
```

---

## FAQ

### Is the Application URL the same as the API URL?

**Yes.** Use the same base URL and append `/api/...`. You do not need a separate API domain (optional: custom subdomain like `api.yourdomain.com` pointing to the same Azure app).

### Will the legacy dashboard clash with our frontend?

**No.** Both can run in parallel. Your app uses `/api/*` only. The dashboard at `/` is independent.

### Do we need Supabase credentials?

**No**, if you use these REST APIs. The backend reads/writes Supabase. Give developers only the Application URL + this doc.

### How do we limit calls per client?

Not built-in yet. Options: (1) enforce limits on your server before calling `/api/call`, or (2) add API keys + quotas to this backend.

### Can we call `/api/call` from the browser?

Possible but not recommended. Use your server as a proxy to avoid CORS and to add auth/limits.

### What happens after `POST /api/call` returns?

The API returns immediately with `"status": "dispatched"`. The actual phone call runs asynchronously. Check `GET /api/calls` or `GET /api/stats` for results after the call completes.

---

## Support checklist for integrators

- [ ] Base URL configured in server environment
- [ ] `GET /api/health` returns OK
- [ ] Test call with valid E.164 number
- [ ] CORS configured if browser → API direct
- [ ] Call limits / auth planned for production
- [ ] Do not expose Azure or Supabase secrets in frontend

---

*Document generated for Swaryx Real Estate OutboundAI backend.*